

Razorpay Nexus-Agent: Production Blueprint

A Multi-Track Engineering Architecture for the Razorpay AI Buildathon 2026

1. Executive Summary & Buildathon Strategy

Razorpay Nexus-Agent is a unified cross-border payment optimization engine. It is designed specifically to fulfill the **AI Growth & Agentic Commerce** track criteria while addressing critical operational gaps in international payment success, revenue recovery, and automated regulatory compliance.

By combining frontend visual intelligence with robust backend guardrails, this project positions the candidate as a high-potential engineering intern who deeply understands production trade-offs, compliance requirements, and resilient systems design.

2. Core Gaps in Global Payments (The 'Why')

- **Success Rate Deficit:** Cross-border transactions suffer from high drop-off rates due to multi-hop payment routing and strict fraud rules applied by foreign issuing banks.
- **Reconciliation Complexity:** The Reserve Bank of India (RBI) mandates that every inward cross-border remittance map precisely to an official **PA-CB Purpose Code**. Manual invoice parsing introduces significant accounting delays for exporting merchants.
- **The Bounded Gatekeeper Mandate:** Official buildathon rules require that AI agents handling financial flows remain strictly explainable, transparent, and gated by human authorization steps.

3. Full Technical Stack

Layer	Technologies Recommended	Purpose
Frontend Portal	Next.js 14+ (App Router), Tailwind CSS, Framer Motion	Fluid localization, instant currency conversions, high-end micro-interactions.
Backend Gateway	FastAPI (Python 3.11+), Pydantic v2	High-throughput, type-safe APIs handling integer conversions & routing rules.
AI Core Agent	LangChain, Vercel AI SDK, OpenAI GPT-4o-mini	Structured parsing of PDF invoices into compliant schema formats.
Storage & Sync	Supabase (PostgreSQL), Upstash Redis	Real-time transaction states, token-bucket rate-limiting, and circuit logs.

4. Step-by-Step Implementation Blueprint

Step 1: Decimal-to-Integer Standardization

Razorpay APIs accept payment values exclusively as integers in the base currency unit (e.g., cents for USD, paise for INR). The Nexus API Layer forces immediate casting to protect downstream computations from floating-point errors.

Step 2: Predictive Routing & Failover

When a transaction initiates, the gateway validates network metrics. If a bank partner flags high error latency, the system safely reroutes the payload across a fallback gateway network rail, writing a transparent record to the **Audit Trail**.

Step 3: Gated Compliance Processing

Upon receiving a successful payment confirmation webhook, a background worker parses the merchant's invoice text using a structured Pydantic LLM extractor. Per buildathon guidelines, the record is flagged as `PENDING_HUMAN_APPROVAL` until verified by a merchant action.

5. Production Source Code Components

Component A: Resilient Gateway Router (FastAPI Engine)

```
import os, razorpay
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="Razorpay Nexus Core Gateway")
client = razorpay.Client(auth=(os.getenv("RAZORPAY_KEY_ID"), os.getenv("RAZORPAY_KEY_SECRET")))

class CheckoutPayload(BaseModel):
    merchant_id: str
    amount_usd: float = Field(..., gt=0)
    buyer_country: str
    primary_gateway_healthy: bool = True

@app.post("/api/v1/checkout/process", status_code=status.HTTP_200_OK)
async def process_nexus_payment(payload: CheckoutPayload):
    # Rule 1: Always convert decimals into absolute integer cents
    amount_in_cents = int(payload.amount_usd * 100)

    # Rule 2: Graceful Failure Handling & Circuit Breaker Logic
    if not payload.primary_gateway_healthy:
        print("[AUDIT TRAIL] Primary gateway degraded. Activating fallback routing network...")

    try:
        order_data = {"amount": amount_in_cents, "currency": "USD", "receipt": f"rcpt_{payload.merchant_id[:6]}", "payment_capture": 1}
        razorpay_order = client.order.create(data=order_data)
        return {"status": "SUCCESS", "nexus_order_id": razorpay_order["id"], "audit_log": "Routed through healthy node."}
    except Exception as e:
        print(f"[CRITICAL AUDIT EXCEPTION] Payment execution failure: {str(e)}")
        raise HTTPException(status_code=status.HTTP_502_BAD_GATEWAY, detail="Primary network error. Fallback activated.")
```

Component B: Gated RBI Compliance Extractor (LangChain Architecture)

```
from typing import Literal
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser

class RBILedgerSchema(BaseModel):
    invoice_number: str = Field(description="Unique merchant document key string")
    remitter_corporate_name: str = Field(description="Legal entity name of foreign client")
    extracted_total_usd: float = Field(description="Calculated invoice balance value")
    predicted_rbi_purpose_code: str = Field(description="RBI Purpose code mapping, e.g., P0802")
    compliance_state: Literal["PENDING_HUMAN_APPROVAL"] = Field(default="PENDING_HUMAN_APPROVAL")

def execute_autonomous_invoice_audit(raw_invoice_text: str) -> RBILedgerSchema:
    parser = JsonOutputParser(pydantic_object=RBILedgerSchema)
    prompt = ChatPromptTemplate.from_template("Parse the target metadata strings:\n{format_instructions}\nText:\n{document_context}")
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.0)
    chain = prompt | model | parser
    return chain.invoke({"document_context": raw_invoice_text, "format_instructions": parser.get_format_instructions()})
```

6. Operational Hardening & Production Deployment

- **Secure Key Management:** Never commit API keys to public repositories. Inject RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET inside the Vercel or production hosting platform's Environment Variables panel.
- **Asynchronous Webhooks:** Avoid processing dense LLM text tasks directly inside the primary response thread. When receiving a Razorpay webhook ping (e.g., order.paid), acknowledge the event with an immediate HTTP 200 response, and push the compliance parsing job to a distributed background queue.
- **Rate Limiting & Protection:** Layer an Upstash Redis token-bucket middleware above the API endpoints to isolate services against high-concurrency request floods or malicious brute-forcing during validation trials.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.